# TECNOLOGÍAS Y PATRÓN DE DESARROLLO

SECUREBANK — Proyecto Final Integrador

**Proyecto 1 · Plataforma FinTech de banca digital segura**

Apartado propuesto para incorporar al plan de trabajo del equipo de 4 integrantes.

**Nota:** El documento original del curso define las tecnologías obligatorias y los componentes que debe integrar SecureBank, pero no establece un lenguaje ni un patrón arquitectónico obligatorio. Las decisiones de este apartado son la propuesta técnica del equipo para cumplir esos requisitos de forma manejable y coherente.

## 1. Lenguaje principal

**TypeScript** será el lenguaje principal de desarrollo. Se utilizará en el backend, frontend web, aplicación móvil y pruebas automatizadas donde la herramienta lo permita.

La principal ventaja para el equipo es mantener un ecosistema de programación homogéneo y reducir la cantidad de lenguajes que los cuatro integrantes deben dominar.

| Componente | Tecnología |
|---|---|
| Lenguaje principal | TypeScript |
| Backend | Node.js + NestJS |
| Frontend Web | React + Vite |
| Aplicación móvil | React Native + Expo |
| Base de datos | PostgreSQL |
| ORM | Prisma |
| Cache | Redis |
| Mensajería | RabbitMQ |
| API | REST |
| Testing API | Insomnia |
| Testing Web | Cypress |
| Testing Mobile | Maestro |
| Testing unitario | Jest |
| Performance | k6 |
| Contenedores | Docker + Docker Compose |
| Cloud | Microsoft Azure |
| IaC | Terraform |
| Configuración | Ansible |
| CI/CD | Jenkins |
| Calidad | SonarQube |
| SAST | Fortify |
| SCA | OWASP Dependency-Check + npm audit |
| DAST | OWASP ZAP |
| Accesibilidad | Lighthouse / axe |

## 2. Patrón de desarrollo recomendado

**Arquitectura:** Monolito Modular + principios de Clean Architecture.

**Patrón interno:** Controller → Service → Repository.

**Principios:** separación de responsabilidades y SOLID.

La propuesta mantiene un solo backend desplegable, pero dividido en módulos funcionales independientes. Esto permite organizar el sistema sin introducir la complejidad operacional de microservicios.

Para un equipo de cuatro personas, esta alternativa permite concentrarse en los requisitos que sí exige el proyecto: API, PostgreSQL, Redis, RabbitMQ, frontend, móvil, Docker, Cloud, IaC, CI/CD, pruebas y seguridad.

## 3. Arquitectura general

```
SECUREBANK
■
■■■ Frontend Web — React + TypeScript
■
■■■ Mobile — React Native + TypeScript
■
■■■ Backend API — NestJS + TypeScript
    ■
    ■■■ Auth
    ■■■ Users
    ■■■ Accounts
    ■■■ Beneficiaries
    ■■■ Transfers
    ■■■ Payments
    ■■■ Audit
    ■■■ Redis
    ■■■ RabbitMQ
        ■
        ■■■ PostgreSQL
        ■■■ Redis
        ■■■ RabbitMQ
```

## 4. Estructura propuesta del backend

```
backend/
■■■ src/
■  ■■■ auth/
■  ■  ■■■ controller/
■  ■  ■■■ service/
■  ■  ■■■ repository/
■  ■  ■■■ dto/
■  ■  ■■■ entities/
■  ■■■ users/
■  ■■■ accounts/
■  ■■■ beneficiaries/
■  ■■■ transfers/
■  ■  ■■■ controller/
■  ■  ■■■ service/
■  ■  ■■■ repository/
■  ■  ■■■ dto/
■  ■  ■■■ events/
■  ■■■ payments/
■  ■■■ audit/
■  ■■■ redis/
■  ■■■ rabbitmq/
■  ■■■ common/
■■■ prisma/
■■■ test/
■■■ package.json
```

## 5. Controller → Service → Repository

**Controller:** recibe las solicitudes HTTP, valida la entrada inicial y devuelve la respuesta.

**Service:** contiene las reglas de negocio. En transferencias valida saldo, autorización, idempotencia y coordina la operación.

**Repository:** concentra el acceso a PostgreSQL mediante Prisma.

El objetivo es evitar que un único archivo mezcle HTTP, reglas bancarias y acceso directo a la base de datos.

## 6. Ejemplo: flujo de una transferencia

```
Usuario
↓
Frontend / Mobile
↓
POST /api/transfers
↓
TransferController
↓
TransferService
■■■ Validar JWT
■■■ Validar autorización
■■■ Validar saldo
■■■ Validar idempotencia
■■■ Ejecutar transacción
■■■ Registrar auditoría
■■■ Publicar evento
↓
TransferRepository → Prisma → PostgreSQL
↓
Redis / RabbitMQ según la operación
```

Este flujo se relaciona directamente con el reto técnico principal de SecureBank: transferencias concurrentes, doble envío, idempotencia, duplicidad y consistencia del saldo.

## 7. Redis y RabbitMQ dentro de la arquitectura

**PostgreSQL:** persistencia de usuarios, cuentas, saldos, transacciones, beneficiarios, pagos y auditoría.

**Redis:** sesiones, rate limiting, tokens temporales y cache, incluyendo pruebas de HIT, MISS, TTL, invalidación y comportamiento ante caída.

**RabbitMQ:** procesamiento de eventos asíncronos como TransferCreated, TransferCompleted, TransferRejected, PaymentCreated y FraudAlert, con consumidor y manejo de retry/DLQ o equivalente.

Los tres servicios se utilizan desde el backend modular; no se crean microservicios separados solamente por tener estas tecnologías.

## 8. Frontend Web

```
frontend/
■■■ src/
■  ■■■ components/
■  ■■■ pages/
■  ■■■ layouts/
■  ■■■ services/
■  ■■■ hooks/
■  ■■■ contexts/
■  ■■■ types/
■  ■■■ utils/
■  ■■■ routes/
■■■ package.json
```

Flujo recomendado: Página → componente/estado → servicio de API → Backend.

## 9. Aplicación móvil

```
mobile/
■■■ src/
■  ■■■ screens/
■  ■■■ components/
■  ■■■ services/
■  ■■■ hooks/
■  ■■■ navigation/
■  ■■■ contexts/
■  ■■■ types/
■■■ package.json
```

La aplicación móvil se comunicará únicamente con la API. No tendrá conexión directa a PostgreSQL.

## 10. Metodología de trabajo del equipo

**Metodología propuesta: Scrum.**

Scrum se utilizará para organizar el trabajo de los cuatro integrantes mediante tareas, prioridades, revisiones y entregas incrementales.

**Control de código:** Git + GitHub + ramas + Pull Requests + Issues.

Cada cambio deberá pasar por implementación, prueba, evidencia y revisión antes de integrarse a la rama principal.

## 11. ¿Por qué no microservicios?

No se recomienda introducir microservicios en este proyecto porque el documento no los exige y aumentarían considerablemente la complejidad de desarrollo, despliegue, comunicación entre servicios, pruebas y diagnóstico.

El Monolito Modular permite demostrar los componentes obligatorios sin perder separación interna: autenticación, cuentas, beneficiarios, transferencias, pagos, auditoría, Redis y RabbitMQ pueden mantenerse como módulos claramente

separados dentro de un mismo backend.

## 12. Decisión técnica final

**Lenguaje:** TypeScript
**Backend:** Node.js + NestJS
**Frontend:** React + Vite
**Mobile:** React Native + Expo
**BD:** PostgreSQL + Prisma
**Arquitectura:** Monolito Modular
**Principios:** Clean Architecture + SOLID
**Patrón interno:** Controller → Service → Repository
**Metodología:** Scrum
**Versionamiento:** Git + GitHub + Pull Requests
**Infraestructura:** Docker + Azure + Terraform + Ansible
**CI/CD:** Jenkins
**QA:** Insomnia + Jest + Cypress + Maestro + k6
**DevSecOps:** SonarQube + Fortify + Dependency-Check + npm audit + OWASP ZAP

Esta selección busca reducir la curva de aprendizaje del equipo y mantener una arquitectura suficientemente estructurada para cumplir los requisitos de calidad, seguridad, automatización, infraestructura y pruebas del proyecto.